NixOS Single-Host Kubernetes Virtualization and DMZ Isolation

Concept document and implementation guide

Status	Proposed target architecture
Scope	NixOS host, libvirt/KVM, K3s VMs, VLAN-isolated DMZ, ZFS-backed storage, Samba NAS
Prepared	2026-07-16
Important	Reference design. Validate option names and behavior against the server's pinned nixpkgs release before deployment.

Core decision: Keep NixOS as the bare-metal NAS and hypervisor. Run every Kubernetes cluster in a full KVM virtual machine. Put trusted internal VMs directly on the LAN bridge and put the public cluster on a dedicated DMZ VLAN and bridge.

Primary security invariant: The NixOS host has no IP address on br-dmz, does not route between the LAN and DMZ, and drops every host-bound IPv4 or IPv6 packet arriving through br-dmz before normal service firewall rules are evaluated.

Contents

1. High-level overview
2. Goals, assumptions, and non-goals
3. Proposed architecture
4. Security model
5. Storage architecture
6. Power and resource considerations
7. NixOS implementation guide
8. Virtual-machine configuration
9. Migration plan
10. Validation and ongoing operations
11. Risks and limitations

Appendix A. Consolidated NixOS reference module

Appendix B. Declarative Mermaid diagram

References

1. High-level overview

The server continues to run NixOS directly on the hardware and remains responsible for ZFS, Samba, networking, and virtual-machine lifecycle management. Kubernetes no longer runs directly on the host. Instead, each single-node K3s cluster runs in a separate KVM virtual machine managed through libvirt.

The architecture uses two security zones. VLAN 10 is the trusted LAN and management network. Internal Kubernetes VMs are normal LAN members. VLAN 30 is the DMZ. The DMZ VM is reachable from the Internet only through router/firewall rules that publish the required application ports, normally TCP 80 and 443.

This provides a substantially clearer security boundary than running K3s directly on the NAS host. Kubernetes networking, container workloads, kernel-facing container operations, and the public ingress stack are contained inside guest kernels rather than sharing the NixOS host kernel.

> **Why NixOS rather than Proxmox:** NixOS with libvirt uses the same QEMU/KVM isolation primitives and Linux bridge/VLAN mechanisms as a dedicated KVM appliance. Proxmox improves convenience and reduces configuration effort, but it does not create a fundamentally stronger VM or VLAN boundary. NixOS is a good fit because the host must remain a customized ZFS and Samba NAS.

2. Goals, assumptions, and non-goals

2.1 Goals

- **Strong public/private separation.** A compromise of an Internet-facing Kubernetes workload should not provide a legitimate management or storage path to the NixOS host or LAN.
- **Retain NAS functionality.** ZFS datasets and Samba remain directly managed by NixOS.
- **Retain dynamic PVC provisioning.** Trusted internal clusters continue to use host ZFS through democratic-csi, now as remote storage.
- **Fail closed.** A service that accidentally binds to 0.0.0.0 or [::] must still be unreachable from the DMZ.

- **Low overhead.** Avoid an additional storage network, distributed storage system, or unnecessary management appliance.
- **Declarative operation.** Network, firewall, Samba, libvirt, and safety checks are expressed in NixOS configuration.

2.2 Assumptions

- The managed switch supports IEEE 802.1Q tagged VLAN trunks.
- The router/firewall owns the VLAN gateway interfaces and performs all inter-VLAN routing.
- The server switch port carries tagged VLAN 10 and VLAN 30.
- The internal Kubernetes VMs and ordinary LAN are considered trusted enough to share a Layer-2 domain.
- The DMZ VM is treated as potentially hostile after compromise.
- The host uses a fixed LAN address, illustrated as 192.168.10.2/24.

2.3 Non-goals and accepted limitations

- This is not a high-availability platform. The physical server remains a single failure domain.
- A single-node Kubernetes cluster is unavailable when its VM or host is down.
- VLANs do not protect against a compromised hypervisor or a successful QEMU/KVM escape.
- Internal VMs are not isolated from the broader LAN by the router because they share br-lan.
- Host ZFS snapshots are not automatically application-consistent database backups.

3. Proposed architecture

3.1 Logical architecture

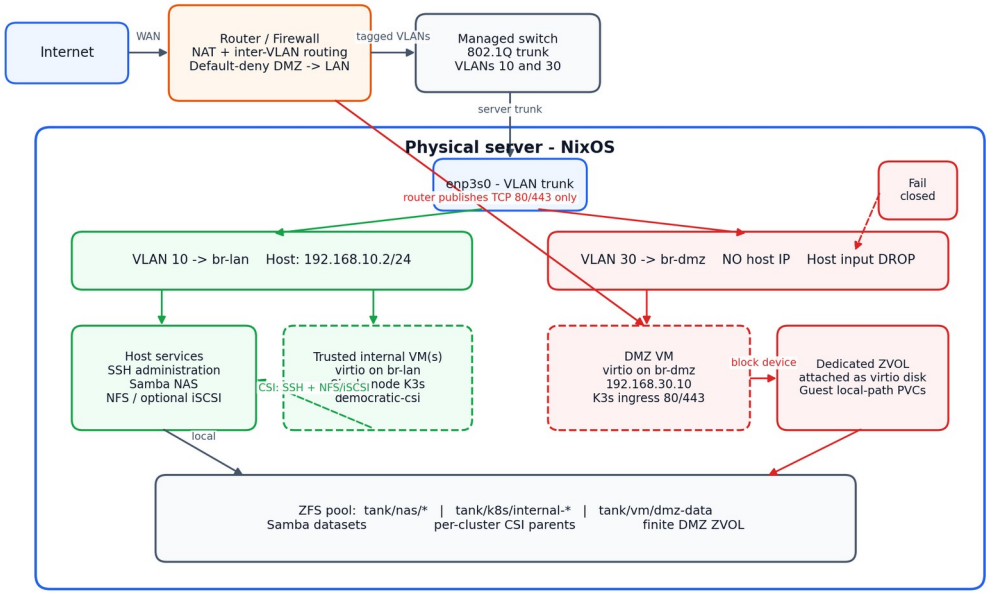


Figure 1 - Proposed network, host, VM, and storage architecture

3.2 Addressing and bridge plan

Component	Example	Purpose / rule
VLAN 10 / br-lan	192.168.10.0/24	Trusted LAN, host management, Samba,

Component	Example	Purpose / rule
		internal VMs, NFS/iSCSI and CSI SSH.
NixOS host	192.168.10.2	Only normal host address. No host address exists in the DMZ.
Internal K3s VM	192.168.10.20+	One virtio NIC attached to br-lan; fixed DHCP reservation or static address.
VLAN 30 / br-dmz	192.168.30.0/24	Public-zone Layer-2 path between router and DMZ VM.
DMZ K3s VM	192.168.30.10	One virtio NIC attached only to br-dmz.
Router gateways	.1 in each subnet	NAT, Internet publication and default-deny inter-VLAN policy.

3.3 Responsibility split

Layer	Responsible component
Internet publication	Router/firewall: NAT or reverse proxy to the DMZ ingress, normally TCP 80/443 only.
Inter-VLAN enforcement	Router/firewall: deny DMZ to LAN by default; allow only explicit administrative flows.
Host exposure	NixOS nftables: unconditional drop on br-dmz plus source/interface-specific LAN rules.
Layer-2 attachment	systemd-networkd Linux bridges and libvirt virtio interfaces.
Workload isolation	Full KVM virtual machines with independent guest kernels.
NAS and storage	NixOS ZFS, Samba, NFS and optional iSCSI.

4. Security model

4.1 Defense-in-depth layers

1. The router/firewall rejects unsolicited Internet traffic except explicitly published DMZ application ports.
2. The router/firewall denies DMZ-to-LAN traffic by default.
3. br-dmz and vlan-dmz have no IPv4, IPv6, DHCP, router-advertisement or link-local address on the host.
4. An early nftables input hook drops every host-bound packet whose input interface is br-dmz.
5. Normal NixOS firewall rules open host services only on br-lan and only from approved source addresses.
6. Samba and SSH are additionally bound to the LAN address/interface where practical.
7. The DMZ VM receives no CSI credentials, NFS/iSCSI access, host filesystem sharing, or libvirt management access.

4.2 Why wildcard listeners are manageable

A listener shown by ss as 0.0.0.0:PORT or [::]:PORT accepts connections on every local address. This is undesirable when a host has an address in an untrusted network. In this design, br-dmz has no host address and the firewall unconditionally drops packets arriving through it. Therefore, an accidental wildcard bind does not by itself expose the service to the DMZ.

Enforcement hierarchy: Service binding is defense in depth. The DMZ input drop is the primary invariant. A future service module may enable a wildcard listener or add a normal firewall allowance; neither should override the earlier br-dmz drop.

4.3 Internal LAN trust trade-off

Removing br-storage and the separate internal VLAN makes the design simpler. Internal VMs can contact the host directly through br-lan, and their traffic normally stays inside the Linux bridge rather than leaving the physical server. The cost is that the router cannot filter communication between the internal VMs and host because they are in the same subnet.

The NixOS host firewall must consequently restrict SSH, NFS, iSCSI and Samba by input interface and source IP. Fixed VM addresses are strongly recommended. Libvirt nwfilter or static DHCP/MAC controls can be added to reduce IP/MAC spoofing risk, but they are optional complexity rather than a core requirement.

4.4 Libvirt and VM hardening

- Use full QEMU/KVM VMs, not host-kernel containers, for every Kubernetes cluster.
- Run QEMU guests as the unprivileged qemu-libvirtd account after confirming disk/device permissions.
- Do not attach host directories to the DMZ VM through virtiofs, 9p, bind mounts, or shared Samba/NFS storage.
- Do not pass USB devices, GPUs, NAS disks, or other host devices to the DMZ VM unless specifically required and reviewed.
- Do not expose libvirt TCP, D-Bus, SSH proxying, or the system libvirt socket to the DMZ.
- Limit VM memory and vCPU allocations to prevent a guest from exhausting host resources.

5. Storage architecture

5.1 ZFS dataset layout

```
tank/
├── nas/
│   └── ...                          # Samba datasets
├── k8s/
│   ├── internal-prod/               # democratic-csi parent dataset
│   ├── internal-prod-snapshots/     # detached snapshots, if used
│   ├── dev/                         # separate cluster and quota boundary
│   └── dev-snapshots/
└── vm/
    ├── internal-root                # VM system disk(s)
    ├── dmz-root                     # DMZ system disk
    └── dmz-data                     # dedicated ZVOL for DMZ PVC data
```

5.2 Trusted internal clusters

The local ZFS democratic-csi driver cannot manage the host pool after Kubernetes moves into a VM because “local” then refers to the guest. Internal clusters should switch to zfs-generic-nfs or, where block storage is needed, zfs-generic-iscsi. The CSI controller connects to the NixOS host over SSH and publishes the created dataset or ZVOL over NFS or iSCSI.

- Use one dedicated SSH key and service account per cluster.
- Use one non-overlapping parent dataset and snapshot parent per cluster.
- Set quotas on each cluster parent to prevent pool exhaustion.

- Restrict host firewall access to the fixed VM source addresses.
- Prefer NFSv4-only operation if practical, so the firewall can expose TCP 2049 without the broader NFSv3 RPC port set.
- Use zfs-generic-nfs for ordinary filesystem PVCs and ReadWriteMany use cases; use iSCSI only when block semantics justify the extra complexity.

5.3 DMZ cluster

The DMZ cluster should not receive credentials capable of creating or destroying host ZFS datasets and exports. Instead, attach a fixed-size ZVOL to the VM as a raw virtio block disk. Format the disk inside the guest and point the K3s local-path provisioner at its mount point.

- The guest can destroy only the data inside its assigned virtual disk through legitimate interfaces.
- The host can snapshot and replicate the ZVOL independently of the VM root disk.
- A finite ZVOL size prevents a public workload from consuming the entire ZFS pool.
- Application-consistent backups still require database-native backup or quiescing procedures.

5.4 Backup model

- **Internal PVCs.** Use ZFS snapshots and replication per CSI-created dataset/ZVOL. Maintain application-level backups for databases.
- **DMZ data disk.** Use guest-agent filesystem freeze plus a host ZFS snapshot where possible; also keep application-level backups.
- **Host configuration.** Store NixOS configuration, libvirt XML, router rules, switch configuration, CSI values and restore procedures in version control and an off-host backup.
- **Samba data.** Keep normal NAS snapshots and replication independent of VM backup schedules.

6. Power and resource considerations

NixOS with libvirt has no inherent power disadvantage compared with Proxmox because both rely on QEMU/KVM and Linux networking. The dominant consumption comes from the physical platform, drives, RAM modules, workload activity and the number of continuously running Kubernetes control planes.

- Use as few independent clusters as operationally necessary; every cluster duplicates the Kubernetes control plane and core add-ons.
- Overcommit vCPU conservatively, but do not heavily overcommit RAM.
- Use CPU type host for efficient virtualization and retain CPU C-states and frequency scaling.
- Prefer SSD/NVMe and avoid distributed storage systems such as Ceph or Longhorn on this single low-power host.
- Avoid a discrete GPU and unnecessary NICs or spinning disks.
- Measure at the wall; idle behavior varies substantially between hardware platforms and firmware settings.

7. NixOS implementation guide

Version note: The following snippets target a recent NixOS generation using systemd-networkd and the nftables firewall backend. Option schemas evolve. Validate with nixos-option, nix eval, and nixos-rebuild build/test against the exact pinned nixpkgs revision.

7.1 Common parameters

```
{ config, lib, pkgs, ... }:

let
  trunkInterface = "enp3s0";

  lanVlanId = 10;
  dmzVlanId = 30;

  hostLanAddress = "192.168.10.2/24";
  lanGateway = "192.168.10.1";

  adminHosts = [ "192.168.10.10" ];
  k8sStorageClients = [
    "192.168.10.20"
    "192.168.10.21"
  ];
in
{ ... }
```

7.2 systemd-networkd: VLANs and bridges

The physical interface is a pure tagged trunk. VLAN subinterfaces are enslaved to separate Linux bridges. The host address and default route exist only on br-lan. br-dmz is intentionally unaddressed.

```
networking.useNetworkd = true;
networking.useDHCP = false;
networking.networkmanager.enable = false;

systemd.network = {
  enable = true;
  wait-online.ignoredInterfaces = [ "br-dmz" ];

  netdevs = {
    "10-vlan-lan" = {
      netdevConfig = {
        Name = "vlan-lan";
        Kind = "vlan";
      };
      vlanConfig.Id = lanVlanId;
    };

    "10-vlan-dmz" = {
      netdevConfig = {
        Name = "vlan-dmz";
        Kind = "vlan";
      };
      vlanConfig.Id = dmzVlanId;
    };

    "20-br-lan" = {
      netdevConfig = {
        Name = "br-lan";
        Kind = "bridge";
      };
      bridgeConfig.STP = false;
    };

    "20-br-dmz" = {
      netdevConfig = {
        Name = "br-dmz";
        Kind = "bridge";
      };
      bridgeConfig.STP = false;
    };
  };

  networks = {
    "10-physical-trunk" = {
      matchConfig.Name = trunkInterface;
      networkConfig = {
        VLAN = [ "vlan-lan" "vlan-dmz" ];
        DHCP = "no";
        LinkLocalAddressing = "no";
        IPv6AcceptRA = false;
      };
    };
```

```
    "20-vlan-lan" = {
      matchConfig.Name = "vlan-lan";
      networkConfig = {
        Bridge = "br-lan";
        DHCP = "no";
        LinkLocalAddressing = "no";
        IPv6AcceptRA = false;
      };
    };

    "20-vlan-dmz" = {
      matchConfig.Name = "vlan-dmz";
      networkConfig = {
        Bridge = "br-dmz";
        DHCP = "no";
        LinkLocalAddressing = "no";
        IPv6AcceptRA = false;
      };
      linkConfig.RequiredForOnline = "no";
    };

    "30-br-lan" = {
      matchConfig.Name = "br-lan";
      address = [ hostLanAddress ];
      routes = [{
        Destination = "0.0.0.0/0";
        Gateway = lanGateway;
      }];
      dns = [ lanGateway ];
      networkConfig = {
        DHCP = "no";
        IPv6AcceptRA = false;
      };
    };

    "30-br-dmz" = {
      matchConfig.Name = "br-dmz";
      networkConfig = {
        DHCP = "no";
        ConfigureWithoutCarrier = true;
        LinkLocalAddressing = "no";
        IPv6AcceptRA = false;
      };
      linkConfig.RequiredForOnline = "no";
    };
  };
};
```

7.3 Disable host routing

```
boot.kernel.sysctl = {
  "net.ipv4.ip_forward" = 0;
  "net.ipv6.conf.all.forwarding" = 0;
};
```

The router/firewall, not the NixOS host, owns inter-VLAN routing. Disabling host forwarding prevents accidental creation of a second path between zones.

7.4 libvirt/KVM

```
virtualisation.libvirtd = {
  enable = true;
  qemu.runAsRoot = false;
  allowedBridges = [ "br-lan" "br-dmz" ];

  # Choose the behavior that matches your shutdown policy.
  onShutdown = "shutdown";
};

users.users.<admin-user>.extraGroups = [ "libvirtd" ];
```

allowedBridges primarily controls qemu:///session bridge-helper use. qemu.runAsRoot = false runs guests as qemu-libvirtd, but existing image files and raw ZVOL devices must be accessible to that account. Test permissions before changing a running installation.

7.5 Fail-closed DMZ host guard

Create a separate nftables table with an input hook that executes earlier than the normal NixOS firewall. It drops both IPv4 and IPv6 traffic arriving on br-dmz and destined for the host. It does not block Layer-2 traffic bridged to the DMZ VM.

```
networking.nftables.enable = true;

networking.nftables.tables."dmz-host-guard" = {
  family = "inet";
  content = ''
    chain input {
      type filter hook input priority -100; policy accept;

      # Host traffic from the DMZ is never accepted.
      iifname "br-dmz" counter drop
    }
  '';
};
```

7.6 Normal host firewall rules

Do not put br-lan or br-dmz in trustedInterfaces and do not use global allowedTCPPorts/allowedUDPPorts for these services. Open each service by input interface and source set.

```
networking.firewall = {
  enable = true;
  backend = "nftables";
  trustedInterfaces = [ ];
  allowedTCPPorts = [ ];
  allowedUDPPorts = [ ];
  logRefusedConnections = true;

  extraInputRules = ''
    set admin_hosts {
      type ipv4_addr
      elements = { ${lib.concatStringsSep ", " adminHosts} }
    }

    set k8s_storage_clients {
      type ipv4_addr
      elements = { ${lib.concatStringsSep ", " k8sStorageClients} }
    }

    # SSH administration and democratic-csi command execution.
    iifname "br-lan" ip saddr @admin_hosts tcp dport 22 accept
    iifname "br-lan" ip saddr @k8s_storage_clients tcp dport 22 accept

    # SMB over TCP. Add UDP 137/138 and TCP 139 only if NetBIOS is needed.
    iifname "br-lan" ip saddr 192.168.10.0/24 tcp dport 445 accept

    # NFSv4 only.
    iifname "br-lan" ip saddr @k8s_storage_clients tcp dport 2049 accept

    # Optional iSCSI.
    iifname "br-lan" ip saddr @k8s_storage_clients tcp dport 3260 accept
  '';
};
```

NFS version: If NFSv3 is enabled, rpcbind and additional fixed RPC ports must also be configured and filtered. NFSv4-only is considerably easier to firewall.

7.7 Samba binding

```
services.samba = {
  enable = true;
  openFirewall = false;

  settings.global = {
    interfaces = "lo br-lan";
    "bind interfaces only" = "yes";
  };
};
```

Keep the firewall rule even when Samba is explicitly bound. Binding mistakes and module changes should not create exposure by themselves.

7.8 SSH binding and account separation

```
services.openssh = {
  enable = true;
  openFirewall = false;

  listenAddresses = [{
    addr = "192.168.10.2";
    port = 22;
  }];

  settings = {
    PasswordAuthentication = false;
    KbdInteractiveAuthentication = false;
    PermitRootLogin = "no";
  };

  extraConfig = ''
    Match User csi-internal csi-dev
      AllowTcpForwarding no
      X11Forwarding no
      PermitTTY no
  '';
};
```

Use a dedicated account and public key for each cluster. The account still needs to execute the command set required by democratic-csi. A tightly tested command wrapper or constrained sudo policy is preferable to unrestricted root SSH, but should be derived from the actual driver configuration and tested before production use.

7.9 Assert that br-dmz remains unaddressed

```
systemd.services.assert-br-dmz-unaddressed = {
  description = "Assert that br-dmz has no host IP addresses";
  wantedBy = [ "multi-user.target" ];
  after = [ "systemd-networkd.service" "network-online.target" ];

  serviceConfig.Type = "oneshot";

  script = ''
    if ${pkgs.iproute2}/bin/ip -o address show dev br-dmz       | ${pkgs.gnugrep}/bin/grep -Eq ' inet6? ';
then
      echo "SECURITY ERROR: br-dmz has an IP address" >&2
      ${pkgs.iproute2}/bin/ip address show dev br-dmz >&2
      exit 1
    fi
  '';
};
```

This assertion is an alarm, not an enforcement mechanism. The nftables input drop remains required.

7.10 Optional service-module guardrails

- Set openFirewall = false on services that provide that option.

- Search the evaluated configuration for unexpected allowed ports and trusted interfaces.
- Review ss -lntup after every material configuration change.
- Do not enable socket activation that bypasses an intended ListenAddress without verifying the generated unit.
- Use NixOS assertions to reject accidental configuration of a DMZ host address or trusted DMZ interface.

8. Virtual-machine configuration

8.1 Internal K3s VM network interface

```
<interface type="bridge">
  <mac address="52:54:00:10:00:20"/>
  <source bridge="br-lan"/>
  <model type="virtio"/>
</interface>
```

Give the VM a fixed address such as 192.168.10.20 and no second storage NIC. democratic-csi connects to 192.168.10.2 over the same bridge.

8.2 DMZ VM network interface

```
<interface type="bridge">
  <mac address="52:54:00:30:00:10"/>
  <source bridge="br-dmz"/>
  <model type="virtio"/>
</interface>
```

The DMZ VM must not also have a br-lan interface. Administrative access should traverse an explicitly permitted router/firewall path or a controlled VPN/bastion flow.

8.3 DMZ ZVOL disk

```
# Example only: select the size and volblocksize for the workload.
sudo zfs create -V 200G tank/vm/dmz-data

# Conceptual libvirt disk definition:
<disk type="block" device="disk">
  <driver name="qemu" type="raw" cache="none" io="native" discard="unmap"/>
  <source dev="/dev/zvol/tank/vm/dmz-data"/>
  <target dev="vdb" bus="virtio"/>
</disk>
```

Inside the guest, format and mount the disk, for example at /var/lib/k3s-storage, then configure the K3s local-path provisioner to use that location. Confirm that qemu-libvirtd can access the ZVOL device when unprivileged QEMU is enabled.

8.4 democratic-csi skeleton for an internal cluster

```
driver: zfs-generic-nfs
instance_id: internal-prod

sshConnection:
  host: 192.168.10.2
  port: 22
  username: csi-internal
  privateKey: |-
    # Inject as a Kubernetes Secret; do not place the private key in Nix.

zfs:
  datasetParentName: tank/k8s/internal-prod
  detachedSnapshotsDatasetParentName: tank/k8s/internal-prod-snapshots
  datasetEnableQuotas: true
  datasetEnableReservation: false

nfs:
  shareStrategy: setDatasetProperties
  shareStrategySetDatasetProperties:
    properties:
      sharenfs: "on"
  shareHost: 192.168.10.2
```

Credential boundary: A cluster that possesses this SSH key can invoke the storage operations allowed to the CSI account. Keep separate keys/accounts and parent datasets for each cluster, and never give these credentials to the DMZ cluster.

9. Migration plan

1. Back up the NixOS configuration, router and switch configuration, Kubernetes manifests, CSI values, and critical ZFS datasets.
2. Configure the switch trunk and router VLAN interfaces. Establish default-deny DMZ-to-LAN rules before attaching a DMZ workload.
3. Add br-lan and br-dmz through systemd-networkd. Perform the change from a local console and use nixos-rebuild test first.
4. Verify that the host has only its LAN address and that br-dmz/vlan-dmz are unaddressed.
5. Enable the early DMZ input drop and test it from a temporary DMZ VM before deploying Kubernetes.
6. Enable libvirt and create the first internal K3s VM on br-lan.
7. Deploy zfs-generic-nfs or zfs-generic-iscsi with a dedicated cluster account and parent dataset.
8. Create replacement PVCs and migrate workloads. Use native dump/restore or replication for databases rather than copying live files.
9. Create the DMZ VM on br-dmz, attach a dedicated ZVOL, and deploy only public workloads there.
10. Publish TCP 80/443 through the router/firewall. Keep the Kubernetes API, SSH, kubelet and overlay ports private.
11. Remove or disable the host-level K3s installation after all workloads and data have been validated.
12. Take post-migration snapshots and document the restore process.

10. Validation and ongoing operations

10.1 Network and bridge validation

```
networkctl
networkctl status enp3s0 vlan-lan vlan-dmz br-lan br-dmz
ip -br address
ip route
bridge link
virsh domiflist <vm-name>
```

Expected result: br-lan has the host address; br-dmz and vlan-dmz have no inet or inet6 address; internal VMs attach only to br-lan; the DMZ VM attaches only to br-dmz.

10.2 Listener and firewall audit

```
sudo ss -lntup
sudo nft list ruleset
sudo nft list table inet dmz-host-guard
sudo journalctl -u nftables -u firewall --since today
```

Every wildcard listener should be understood. The dmz-host-guard counter should remain visible and increase when a DMZ system attempts to contact the host.

10.3 Active security tests

- From the DMZ VM, scan the host LAN address for SSH, SMB, NFS and iSCSI. All should appear filtered or unreachable.
- From an approved internal VM, confirm NFS/iSCSI and CSI SSH access.
- From a normal LAN client that is not a storage client, verify that NFS/iSCSI and CSI SSH are denied.
- From the approved administration host, confirm SSH access.
- Confirm the Internet reaches only the intended ingress ports and cannot reach the Kubernetes API, kubelet or Flannel VXLAN ports.
- Repeat IPv6 tests if IPv6 is enabled anywhere in the environment.

10.4 Change management

- Use nixos-rebuild build for syntax and evaluation checks.
- Use nixos-rebuild test before switch for network/firewall changes.
- Keep console or out-of-band access during bridge and firewall work.
- Review the generated nftables ruleset rather than assuming a module option behaved as intended.
- Pin nixpkgs and review release notes before upgrading the host.
- Keep a rollback generation and tested boot path.

11. Risks and limitations

Risk	Mitigation / accepted position
Single physical failure domain	External backups, tested restore, UPS if appropriate; add a second physical host only if availability requirements justify it.
DMZ VM escape	Patch host/QEMU, minimize device passthrough and host sharing, run QEMU unprivileged, maintain backups. Cannot be eliminated on one host.
Compromised internal cluster abuses CSI credentials	Separate accounts/keys/datasets, quotas, source firewalling, constrained sudo or command wrapper, logging and snapshots.
LAN device attacks host services	Source-specific host firewall, VLAN guest/IoT devices separately if they are not trusted, strong SSH and Samba authentication.
ZFS pool exhaustion	Parent quotas, finite DMZ ZVOL, monitoring and alerting, reservation strategy appropriate to the workload.
Application-inconsistent snapshots	Database-native backups, quiescing, guest agent and restore testing.
NixOS option changes	Pin nixpkgs and validate the generated system on the exact

Risk	Mitigation / accepted position
	release before switching.

Appendix A. Consolidated NixOS reference module

The following example combines the main network, virtualization and firewall concepts. It is intentionally a reference skeleton rather than a complete production configuration. Replace addresses, users, shares, keys and storage services; validate all options against the pinned nixpkgs revision.

```
{ config, lib, pkgs, ... }:

let
  trunkInterface = "enp3s0";
  lanVlanId = 10;
  dmzVlanId = 30;

  hostLanAddress = "192.168.10.2/24";
  lanGateway = "192.168.10.1";

  adminHosts = [ "192.168.10.10" ];
  k8sStorageClients = [ "192.168.10.20" "192.168.10.21" ];
in
{
  networking.useNetworkd = true;
  networking.useDHCP = false;
  networking.networkmanager.enable = false;

  systemd.network = {
    enable = true;
    wait-online.ignoredInterfaces = [ "br-dmz" ];

    netdevs = {
      "10-vlan-lan" = {
        netdevConfig = { Name = "vlan-lan"; Kind = "vlan"; };
        vlanConfig.Id = lanVlanId;
      };
      "10-vlan-dmz" = {
        netdevConfig = { Name = "vlan-dmz"; Kind = "vlan"; };
        vlanConfig.Id = dmzVlanId;
      };
      "20-br-lan" = {
        netdevConfig = { Name = "br-lan"; Kind = "bridge"; };
        bridgeConfig.STP = false;
      };
      "20-br-dmz" = {
        netdevConfig = { Name = "br-dmz"; Kind = "bridge"; };
        bridgeConfig.STP = false;
      };
    };

    networks = {
      "10-physical-trunk" = {
        matchConfig.Name = trunkInterface;
        networkConfig = {
          VLAN = [ "vlan-lan" "vlan-dmz" ];
          DHCP = "no";
          LinkLocalAddressing = "no";
          IPv6AcceptRA = false;
        };
      };
      "20-vlan-lan" = {
        matchConfig.Name = "vlan-lan";
        networkConfig = {
          Bridge = "br-lan";
          DHCP = "no";
          LinkLocalAddressing = "no";
          IPv6AcceptRA = false;
        };
      };
      "20-vlan-dmz" = {
        matchConfig.Name = "vlan-dmz";
        networkConfig = {
          Bridge = "br-dmz";
          DHCP = "no";
          LinkLocalAddressing = "no";
          IPv6AcceptRA = false;
        };
        linkConfig.RequiredForOnline = "no";
      };
      "30-br-lan" = {
        matchConfig.Name = "br-lan";
        address = [ hostLanAddress ];
        routes = [{ Destination = "0.0.0.0/0"; Gateway = lanGateway; }];
```

```
      dns = [ lanGateway ];
      networkConfig = { DHCP = "no"; IPv6AcceptRA = false; };
    };
    "30-br-dmz" = {
      matchConfig.Name = "br-dmz";
      networkConfig = {
        DHCP = "no";
        ConfigureWithoutCarrier = true;
        LinkLocalAddressing = "no";
        IPv6AcceptRA = false;
      };
      linkConfig.RequiredForOnline = "no";
    };
  };
};

boot.kernel.sysctl = {
  "net.ipv4.ip_forward" = 0;
  "net.ipv6.conf.all.forwarding" = 0;
};

virtualisation.libvirtd = {
  enable = true;
  qemu.runAsRoot = false;
  allowedBridges = [ "br-lan" "br-dmz" ];
  onShutdown = "shutdown";
};

users.users.<admin-user>.extraGroups = [ "libvirtd" ];

networking.nftables.enable = true;
networking.nftables.tables."dmz-host-guard" = {
  family = "inet";
  content = ''
    chain input {
      type filter hook input priority -100; policy accept;
      iifname "br-dmz" counter drop
    }
  '';
};

networking.firewall = {
  enable = true;
  backend = "nftables";
  trustedInterfaces = [ ];
  allowedTCPPorts = [ ];
  allowedUDPPorts = [ ];
  logRefusedConnections = true;

  extraInputRules = ''
    set admin_hosts {
      type ipv4_addr
      elements = { ${lib.concatStringsSep ", " adminHosts} }
    }
    set k8s_storage_clients {
      type ipv4_addr
      elements = { ${lib.concatStringsSep ", " k8sStorageClients} }
    }
```

```
    iifname "br-lan" ip saddr @admin_hosts tcp dport 22 accept
    iifname "br-lan" ip saddr @k8s_storage_clients tcp dport 22 accept
    iifname "br-lan" ip saddr 192.168.10.0/24 tcp dport 445 accept
    iifname "br-lan" ip saddr @k8s_storage_clients tcp dport 2049 accept
    # iifname "br-lan" ip saddr @k8s_storage_clients tcp dport 3260 accept
  '';
};

services.samba = {
  enable = true;
  openFirewall = false;
  settings.global = {
    interfaces = "lo br-lan";
    "bind interfaces only" = "yes";
  };
};

services.openssh = {
  enable = true;
  openFirewall = false;
  listenAddresses = [{ addr = "192.168.10.2"; port = 22; }];
  settings = {
    PasswordAuthentication = false;
    KbdInteractiveAuthentication = false;
    PermitRootLogin = "no";
  };
};

systemd.services.assert-br-dmz-unaddressed = {
  description = "Assert that br-dmz has no host IP addresses";
  wantedBy = [ "multi-user.target" ];
```

```
    after = [ "systemd-networkd.service" "network-online.target" ];
    serviceConfig.Type = "oneshot";
    script = ''
      if ${pkgs.iproute2}/bin/ip -o address show dev br-dmz \
        | ${pkgs.gnugrep}/bin/grep -Eq ' inet6? '; then
        echo "SECURITY ERROR: br-dmz has an IP address" >&2
        ${pkgs.iproute2}/bin/ip address show dev br-dmz >&2
        exit 1
      fi
    '';
  };
}
```

Appendix B. Declarative Mermaid diagram

```
flowchart TB
  Internet((Internet)) --> Router[Router / firewall]
  Router -->|VLAN 10: LAN| Switch[Managed switch]
  Router -->|VLAN 30: DMZ| Switch
  Switch -->|802.1Q trunk| NIC[enp3s0 on NixOS]

  subgraph Host[NixOS physical server]
    NIC --> VLAN10[vlan-lan / VLAN 10] --> BRLAN[br-lan\n192.168.10.2/24]
    NIC --> VLAN30[vlan-dmz / VLAN 30] --> BRDMZ[br-dmz\nno host IP\ninput dropped]

    BRLAN --> Services[SSH + Samba + NFS/iSCSI + ZFS + libvirt]
    BRLAN --> InternalVM[Internal K3s VM\n192.168.10.20+]
    BRDMZ --> DMZVM[DMZ K3s VM\n192.168.30.10]

    Services --> Pool[(ZFS pool)]
    InternalVM -. democratic-csi\nSSH + NFS/iSCSI .-> Services
    Pool -->|dedicated ZVOL| DMZVM
  end

  Router -->|NAT / port-forward TCP 80,443| DMZVM
  Router -. deny DMZ to LAN .-> BRLAN
```

Render with Mermaid CLI: mmdc -i architecture.mmd -o architecture.svg

References

[R1] NixOS nftables firewall module (extraInputRules and generated input chain):
https://github.com/NixOS/nixpkgs/blob/master/nixos/modules/services/networking/firewall-nftables.nix

[R2] NixOS libvirt module (allowedBridges and qemu.runAsRoot):
https://github.com/NixOS/nixpkgs/blob/master/nixos/modules/virtualisation/libvirtd.nix

[R3] NixOS systemd-networkd test/configuration examples:
https://github.com/NixOS/nixpkgs/blob/master/nixos/tests/systemd-networkd.nix

[R4] libvirt domain XML: bridge interfaces and disk definitions:
https://libvirt.org/formatdomain.html

[R5] democratic-csi drivers and requirements:
https://github.com/democratic-csi/democratic-csi/blob/master/README.md

[R6] democratic-csi zfs-generic-nfs example:
https://github.com/democratic-csi/democratic-csi/blob/master/examples/zfs-generic-nfs.yaml

[R7] K3s volumes and local-path provisioner: https://docs.k3s.io/add-ons/storage

[R8] systemd.network configuration reference:
https://www.freedesktop.org/software/systemd/man/latest/systemd.network.html

Final implementation rule: Do not deploy the snippets blindly. Build and inspect the generated system, nftables rules, networkd units and libvirt domain XML on the exact NixOS release in use, then test from both the LAN and DMZ before migrating production workloads.